

BTS Services Informatiques aux Organisations (SIO)

Session 2026

Nom du candidat : SEILLER

Prénom du candidat : Julian

Parcours : SLAM

Fiche de réalisation professionnelle N°10

Développement d'une plateforme de suivi de scolarité

(Projet Collège Asimov)

Période de réalisation : 2ème année de BTS

Type de réalisation : En formation (Lycée Louise Michel)

Mode de réalisation : En autonomie

Sommaire

Contexte Organisationnel.....	3
Problématique de gestion.....	3
Choix de la solution mise en œuvre.....	3
Réalisation de la solution.....	4
a - Prérequis et ressources à disposition.....	4
b - Mise en œuvre de la solution.....	4
c - Tests.....	4
i - Cas d'utilisation 1 : Test de la route API de récupération des élèves (GET).....	4
ii - Cas d'utilisation 2 : Alerte sur la recherche de stages. .	5
iii - Cas d'utilisation 3 : Génération de la convention de stage.....	5
Conclusions.....	5
Annexes.....	5
Compétences associées.....	6

Contexte Organisationnel

Cette réalisation a été effectuée au sein du **Lycée Louise Michel** dans le cadre du cours de **SLAM** (Solutions Logicielles et Applications Métier). Le projet consistait à développer une application pour le **Collège Asimov**, un établissement fictif ayant besoin d'un outil moderne pour gérer ses effectifs et les résultats scolaires des élèves.

Problématique de gestion

L'objectif n'était pas de créer un simple logiciel de notes, mais un véritable outil de suivi du potentiel et des initiatives des élèves. Le cahier des charges imposait des défis techniques précis :

- Permettre au secrétariat d'importer massivement les nouveaux élèves de sixième via des fichiers CSV.
- Mettre en place un module de suivi des stages capable de comptabiliser les entreprises contactées et d'alerter les professeurs référents en cas de difficulté.
- Générer automatiquement des conventions de stage au format PDF.
- Garantir une architecture moderne séparant le back-end (API) et le front-end (Client).

Choix de la solution mise en œuvre

Pour répondre aux exigences de la DSI (M. Yohan Livestone), j'ai opté pour une architecture distribuée basée sur l'écosystème JavaScript :

- **Serveur d'API REST** : Développement d'une API centralisée en Node.js fournissant les données au format JSON, documentée avec JSDoc.
- **Base de données** : Utilisation d'une base de données relationnelle (**SQLite**) pour gérer les contraintes d'intégrité entre les élèves, les classes, les parents et les stages.
- **Client Web** : Création d'une application front-end avec Express.js et EJS pour consommer l'API REST et afficher les interfaces utilisateurs (Tableaux de bord, graphiques d'évolution des moyennes).

- **Architecture MVC** : Structuration stricte du code (Modèle-Vue-Contrôleur) pour faciliter le travail collaboratif sur Git.

Réalisation de la solution

a - Prérequis et ressources à disposition

- Serveur Node.js et gestionnaire NPM.
- Bibliothèques de traitement CSV et de génération PDF.
- Dépôt Git en ligne pour le versioning et outil Kanban pour la répartition des tâches (User Stories).

b - Mise en œuvre de la solution

- **Développement de l'API** : J'ai programmé les différentes routes (endpoints) permettant de créer, lire, modifier et supprimer les données des élèves et de leurs recherches de stage.
- **Module d'import CSV** : J'ai développé un script côté serveur capable de lire un fichier CSV fourni par le secrétariat, d'en extraire les colonnes (nom, prénom, identifiant, niveau) et d'insérer automatiquement les élèves en base de données.
- **Suivi des stages** : J'ai créé une vue dédiée pour les professeurs référents. La logique métier calcule le nombre d'entreprises contactées par l'élève et applique un style visuel différent (alerte rouge) si ce nombre dépasse 15 contacts sans succès.
- **Export PDF** : Intégration d'une librairie permettant de fusionner les données de l'élève (nom, entreprise, dates) dans une convention de stage PDF vierge.

c - Tests

i - Cas d'utilisation 1 : Test de la route API de récupération des élèves (GET)

- **Entrée / Action** : Envoi d'une requête GET sur la route /api/eleves via l'outil de test Postman.
- **Résultat attendu** : L'API renvoie un code HTTP 200 et le flux JSON listant l'intégralité des 120 élèves en moins de 100ms.

ii - Cas d'utilisation 2 : Alerte sur la recherche de stages

- **Entrée / Action** : Un élève ajoute sa 16ème tentative de contact d'entreprise (sans réponse positive) dans son espace.
- **Résultat attendu** : Lors de la connexion de son professeur référent, le nom de l'élève apparaît en surbrillance pour déclencher une aide pédagogique.

iii - Cas d'utilisation 3 : Génération de la convention de stage

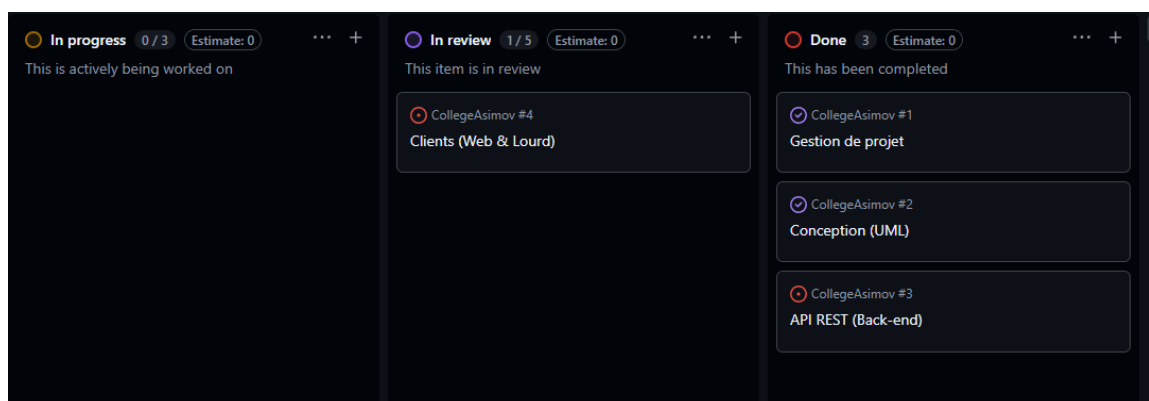
- **Entrée / Action** : L'élève valide les informations de son entreprise d'accueil et clique sur "Générer la convention".
- **Résultat attendu** : Le serveur compile les données et force le téléchargement d'un fichier PDF pré-rempli, prêt à être signé par l'enseignant référent et les parents.

Conclusions

Le projet Asimov a été une excellente simulation des conditions réelles du monde professionnel. J'ai appris à concevoir et documenter une **API REST** robuste, condition indispensable aujourd'hui pour faire communiquer des clients légers (Web) et lourds. La gestion du projet en mode Agile (Kanban) et l'utilisation de Git m'ont permis de coordonner efficacement mes développements avec le reste de l'équipe, tout en respectant les délais et les pénalités fixés par le client.

Annexes

- **Annexe 1** : Capture d'écran de mon tableau Kanban personnel montrant l'avancement des tâches (Backlog / Terminé).



- **Annexe 2 :** Extrait code de « routes/eleves.js ».

```
const express = require('express');
const router = express.Router();
const eleveController = require('../controllers/eleveController');
const { protect, restrictTo } = require('../middleware/authMiddleware');

// Route spécifique : Import CSV (US01) - Doit être déclarée avant /:id pour éviter les conflits de route
router.post('/import', protect, restrictTo('Secretariat', 'Proviseur'), eleveController.importCSV);

// Routes CRUD classiques
router.get('/', protect, eleveController.getAllEleves);
router.get('/:id', protect, eleveController.getEleveById);
router.post('/', protect, restrictTo('Secretariat', 'Proviseur'), eleveController.createEleve);
router.put('/:id', protect, restrictTo('Secretariat', 'Proviseur'), eleveController.updateEleve);
router.delete('/:id', protect, restrictTo('Proviseur'), eleveController.deleteEleve);

module.exports = router;
```

Compétences associées

N°	Compétences associées
B2.1.4	Concevoir et développer une solution applicative (API REST et Client Web)
B1.1.7	Gérer le patrimoine informatique (Déploiement et respect du RGPD)
B1.4.2	Organiser son travail (Méthode Agile, Kanban et Git)
B1.4.1	Analyser les objectifs et les modalités d'organisation d'un projet